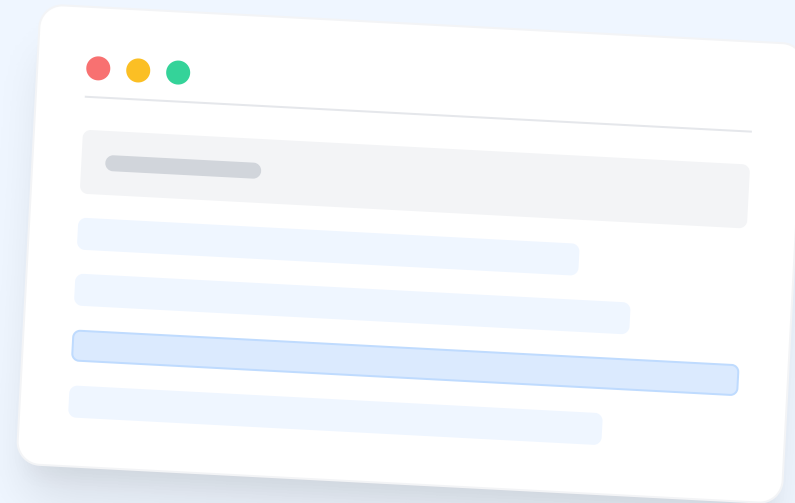


데이터 선택, 필터링 및 정렬

Slicing & Indexing

🎯 이번 강의 핵심 목표

- ✓ 내가 원하는 데이터만 정확하게 **추출하고 조작**한다
- ✓ **loc, iloc** 동작 원리와 차이를 이해한다
- ✓ 조건부 필터링(Boolean Indexing)을 능숙하게 사용한다
- ✓ **컬럼 조작 및 정렬**(sort_values)로 데이터를 정리한다



TODAY'S DATASET

Seaborn Titanic

목차와 시간 계획 (3시간)

진행 방식



개념 설명

비유와 도식으로 원리를 직관적으로 이해합니다.



미니 데모

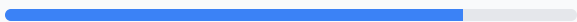
간단한 코드로 즉시 동작을 확인합니다.



실습 과제

타이타닉 데이터로 직접 문제를 해결합니다.

"데이터 분석의 80%는
데이터를 찾고 정리하는 과정입니다."



•



도입: 왜 선택이 중요한가

서랍에서 문서 찾기 비유

•



Section 1. 인덱싱과 슬라이싱

loc vs iloc 완벽 이해

•



Section 2. 조건부 필터링

Boolean Indexing 활용

•



Section 3. 컬럼 조작

추가, 삭제, 이름 변경

•



Section 4. 데이터 정렬

sort_values, sort_index

•

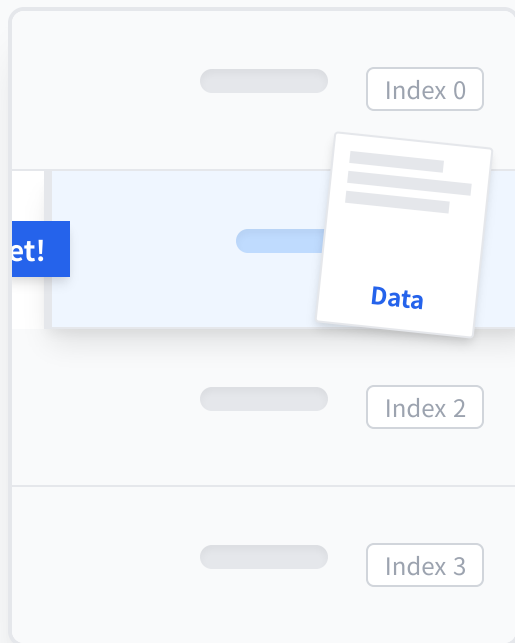


종합 실습·요약·Q&A

타이타닉 생존자 분석 미션

데이터 선택은 '서랍에서 문서 찾기'

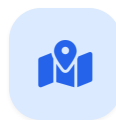
66



**"수천 장의 문서 중
필요한 것만 3초 안에 꺼내기"**

잘못된 서랍(Index)을 열거나
주소(Label)를 모르면 시간을 낭비합니다.

오늘 얻어갈 3가지 핵심 무기



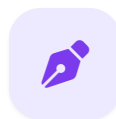
주소 vs 좌표 (loc vs iloc)

"강남구 테헤란로(주소)"로 찾을 것인가,
"위도 37.5, 경도 127.0(좌표)"으로 찾을 것인가?



질문으로 거르기 (Boolean Mask)

"나이가 20세 이상인가?"라는 질문(True/False)으로
원하는 데이터만 남기는 마법의 필터.



정리와 가공 (Edit & Sort)

필요한 새 항목(Column)을 만들고,
보기 좋게 순서(Sort)를 정렬하여 스토리를 완성합니다.

SECTION 01

인덱싱과 슬라이싱

데이터 분석의 첫걸음,
정확한 좌표로 데이터에 접근하는 법을 배웁니다.



학습 목표

이번 섹션에서 다룰 핵심 내용입니다.

1

DataFrame 구조 복습

행(Row), 열(Column), 인덱스(Index)의 관계를 명확히 이해하고 구조를 파악합니다.

2

loc vs iloc 완벽 비교

이름 기반 주소(loc)와 위치 기반 좌표(iloc)의 차이점과 올바른 사용 시점을 익힙니다.

3

선택 기법과 주의사항

단일 값 및 범위 선택 방법을 실습하고, SettingWithCopyWarning 등 체인 인덱싱의 함정을 피합니다.

DataFrame 구조 빠른 복습



| DataFrame의 3대 핵심 요소

1
2

01

Index (행 라벨)

데이터의 각 행을 고유하게 식별하는 주소입니다. 사람으로 치면 주민등록번호와 같습니다.

EXAMPLE

0, 1, 2... / 'row1', 'row2'

00

02

Columns (열 이름)

데이터의 속성이나 변수명을 의미합니다. 가로로 나열된 각 필드의 이름입니다.

EXAMPLE

'age', 'sex', 'fare'

000

03

Values (데이터 값)

실제 저장된 정보의 내용물입니다. 2차원 배열(Matrix) 형태로 저장됩니다.

EXAMPLE

22.0, 'male', 7.25



Shape (크기) 확인하기

데이터 로드 직후 가장 먼저 확인해야 할 속성입니다. 데이터의 규모를 파악하는 첫 단계입니다.

`df.shape` → (행 개수, 열 개수)

Tuple 반환

dataframe_basic.py

```
1 import pandas as pd
2 import seaborn as sns
3
4 # 1. 타이타닉 데이터셋 로드
5 df = sns.load_dataset('titanic')
6
7 # 2. 데이터 형태(Shape) 확인: (행, 열)
8 # 가장 먼저 데이터의 규모를 파악합니다.
9 print(f"Shape: {df.shape}")
10
11 # 3. 인덱스와 컬럼 확인
12 # 행을 식별하는 Index와 열 이름인 Columns를 봅니다.
13 print("Index:", df.index)
14 print("Columns:", df.columns)
15
16 # 4. 상위 데이터 미리보기 (기본 5행)
17 # 데이터가 제대로 로드되었는지 눈으로 직접 확인합니다.
18 df_head = df.head(3) display(df_head)
19
```

OUTPUT / TERMINAL

→ python dataframe_basic.py

Shape: (891, 15)

Index Info:

RangeIndex(start=0, stop=891, step=1)

Columns Info:

Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch',
'fare', 'embarked', 'class', 'who', 'adult_male', 'deck',
'embark_town', 'alive', 'alone'], dtype='object')

DataFrame Head (Top 3):

	survived	pclass	sex	age	fare
0	0	3	'male'	22.0	7.250
1	1	1	'female'	38.0	71.283
2	1	3	'female'	26.0	7.925
... 10 more columns ...					

loc vs iloc: 주소 vs 좌표

접근 방식의 차이

METHOD 1

Label

"서울시 강남구..."

🏠 명시적 주소

METHOD 2

Position

"위도 37.5, 경도 127..."

📊 절대 좌표 (숫자)



loc Label-based Indexing

사람이 이해하기 쉬운 **이름(Label)**을 사용합니다.
행과 열의 이름이 바뀔 때까지 유효합니다.

⚠️ 슬라이싱 끝 포함 (Inclusive): `loc[0:3] → 0, 1, 2, 3`



iloc Integer-based Indexing

컴퓨터 관점의 **순서(Integer Position)**를 사용합니다.
데이터 정렬 순서가 바뀌면 가리키는 대상도 바뀝니다.

⚠️ 슬라이싱 끝 제외 (Exclusive): `iloc[0:3] → 0, 1, 2`

FEATURE	loc	iloc
Input Type	이름 (Label)	정수 (Integer)
Range [A:B]	B 포함	B 제외

체인 인덱싱 (Chained Indexing) 주의

데이터 수정 시 가장 많이 하는 실수

기본 선택 패턴 복습

단일 열 (Series)

```
df['age']
```

다중 열 (DataFrame)

```
df[['sex', 'age']]
```

범위 슬라이싱

```
df.loc[:, 'A':'C']
```

* 조회(Read)할 때는 위 방법들이 문제없습니다.



SettingWithCopyWarning 발생 원인

데이터를 수정(Write)할 때, `df[조건][열]` 처럼 두 단계로 접근하면 Pandas는 원본을 고쳐야 할지, 임시 복사본을 고쳐야 할지 혼동합니다.

체인 인덱싱

```
# [조건] 후 [열] 선택 (2단계)  
df[mask]['col'] = 'X'
```

DataFrame → Copy? → 반영 안 됨!

"복사본에 값을 대입하는 실수"

DON'T

loc 사용

```
# [조건, 열] 한번에 선택 (1단계)  
df.loc[mask, 'col'] = 'O'
```

DataFrame → 원본 수정 완료

"원본 데이터 위치를 정확히 타격"

RECOMMENDED

indexing_comparison.py

```
1 # 1. 단일 데이터 선택: 0번 승객의 'age' 찾기
2 row_idx, col_name, col_pos = 0, 'age', 3
3
4 # loc: [라벨, 라벨] → "주소"로 찾기 (직관적)
5 age_loc = df.loc[row_idx, col_name]
6
7 # iloc: [정수, 정수] → "좌표"로 찾기 (수학적)
8 age_iloc = df.iloc[row_idx, col_pos]
9 print(f"Loc Value: {age_loc} vs Iloc Value: {age_iloc}")
10
11 # 2. 범위 슬라이싱 비교 (가장 중요한 차이!)
12 # loc[0:3]: 0, 1, 2, 3 (끝 포함 0) → 4명 선택
13 slice_loc = df.loc[0:3, 'age']
14
15 # iloc[0:3]: 0, 1, 2 (끝 제외 X) → 3명 선택
16 slice_iloc = df.iloc[0:3, 3]
17 print(f"\nRange Result:")
18 print(f"loc[0:3] count: {len(slice_loc)} (끝 포함)")
19 print(f"iloc[0:3] count: {len(slice_iloc)} (끝 제외)")
```

OUTPUT / TERMINAL

→ python indexing_comparison.py

Loc Value: 22.0 vs Iloc Value: 22.0

Range Slicing Comparison:

print(slice_loc) --> loc[0:3] (4 rows)

```
0 22.0
1 38.0
2 26.0
3 35.0
Name: age, dtype: float64
```

▲ End index '3' is INCLUDED

print(slice_iloc) --> iloc[0:3] (3 rows)

```
0 22.0
1 38.0
2 26.0
Name: age, dtype: float64
```

▲ End index '3' is EXCLUDED

selection_patterns.py

```
1 # 1. 자주 쓰는 선택 패턴 (Selection Patterns)
2 s_age = df['age']
3 # 단일 열 (Series 반환)
4
5 df_sub = df[['sex', 'age']]
6 # 다중 열 (DataFrame 반환)
7
8 df_step = df.iloc[::2, :]
9 # 행 스텝 (짝수 행만 추출)
10
11 # 목표: 60세 초과 승객의 성별을 'senior'로 변경하고 싶다.
12
13 # 2. ❌ 잘못된 방법: 체인 인덱싱 (Chained Indexing)
14 # df[ ... ]로 먼저 자르고, ['sex']로 또 접근하여 값을 할당
15 # df[df['age'] > 60]['sex'] = 'senior' ← 경고 발생!
16
17 # 3. ✅ 올바른 방법: loc 사용
18 # (행 조건, 열 이름)을 loc 안에 한 번에 명시해야 안전합니다.
19 df.loc[df['age'] > 60, 'sex'] = 'senior'
20 print("수정 완료!")
21 print(df.loc[df['age'] > 60, ['age', 'sex']].head(2))
```

OUTPUT / TERMINAL

[→ python selection_patterns.py](#)

Executing "Bad Practice" code...

⚠ SettingWithCopyWarning

A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

Executing "Good Practice" (.loc)...

수정 완료!

age	sex
3366.0	'senior'
5465.0	'senior'

* 데이터가 원본(df)에 정상적으로 반영되었습니다.

실습 1: 인덱싱·슬라이싱 손풀기

GOAL

loc과 iloc을 사용하여 원하는 행·열을 **정확하게 선택** 한다.

1 라벨 기반 선택 (loc)

첫 5개 행에서 **sex**, **age**, **fare** 열만 선택하세요.

2 위치 기반 선택 (iloc)

10행부터 19행까지의 **모든** 열을 선택하세요. (끝 번호 주의!)

3 스텝 슬라이싱 (Step)

전체 데이터에서 **홀수 행**(1, 3, 5...)만 추출하세요. `start:stop:step`

4 새로운 DataFrame 생성

age 와 **fare** 열만 선택하여 **df_sub** 변수에 저장하세요.

exercise_01.py

```
1 df = sns.load_dataset('titanic')
2 # 1. loc 사용 (끝 포함 주의)
3 task1 = df.loc[_____, ['sex', 'age', 'fare']]
4 # 2. iloc 사용 (끝 제외)
5 task2 = df.iloc[10:20, _____]
6 # 3. Step 슬라이싱 [start:end:step]
7 task3 = df.iloc[1::2, ____]
8 # 4. DataFrame 추출
9 df_sub = df[_____]
```

Hint

- **loc**은 이름을 사용하고 끝 범위를 **포함**합니다. (0:5 → 0,1,2,3,4,5)
- **iloc**은 번호를 사용하고 끝 범위를 **제외**합니다. (10:20 → 10~19)
- 모든 행/열을 선택할 땐 콜론(:) 하나만 쓰세요!

SECTION 02

조건부 필터링

Boolean Indexing

데이터 분석의 핵심 기술,
원하는 조건에 맞는 데이터만
정확하게 골라내는 방법을 배웁니다.



학습 목표

이번 섹션에서 다룰 핵심 내용입니다.

1

불리언 마스크(Boolean Mask) 원리

True/False로 구성된 시리즈를 이용해 데이터를 필터링하는 기본 원리를 이해합니다.

2

복합 조건 활용 (&, |, ~)

AND, OR, NOT 연산자를 사용하여 여러 조건을 동시에 만족하는 데이터를 추출합니다.

3

유용한 메서드와 결측치 처리

isin(), between() 등 가독성 높은 메서드와 isna()를 활용한 결측치 필터링을 익힙니다.

Boolean 마스크: True/False로 거르기

필터링의 핵심 3단계



1. 질문 던지기 (Condition)

"나이가 18세 이상인가?"와 같이 예/아니오로 답할 수 있는 조건을 만듭니다.



2. 마스크 생성 (Boolean Mask)

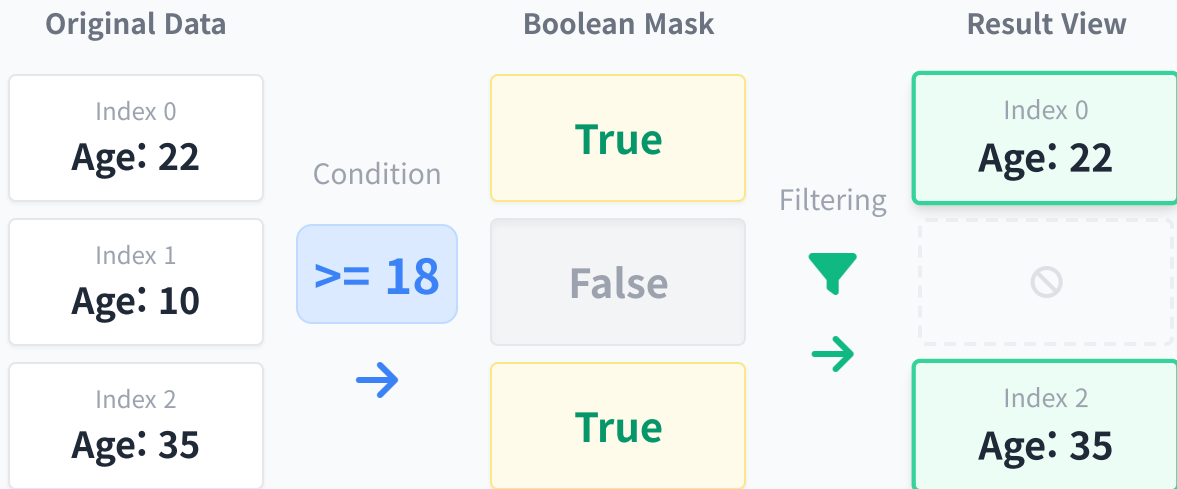
질문의 결과로 데이터 길이와 똑같은 **True/False** 시리즈가 만들어집니다.



3. 데이터 추출 (Indexing)

마스크를 씌우면 **True**인 데이터만 남고 False는 사라집니다.

작동 원리 시각화



i False 위치의 데이터는 삭제되는 것이 아니라 **결과 화면(View)**에서 숨겨지는 것입니다.

조건부 필터링의 논리 구조



&

INTERSECTION



|

UNION



~

COMPLEMENT



판다스의 조건부 필터링은
집합(Set)의 원리와 동일합니다.

복합 조건을 다루는 3가지 원칙



비트 연산자 사용 (Bitwise Operators)

파이썬의 `and`, `or` 대신 기호 연산자 `&`, `|`, `~`를 사용해야 합니다.



괄호 필수 규칙 (Parentheses Rule)

연산자 우선순위 문제로 인해 각 조건은 반드시 괄호로 감싸야 합니다.

```
✗ df['age'] > 20 & df['sex'] == 'f'
✓ (df['age'] > 20) & (df['sex'] == 'f')
```



가독성을 높이는 메서드

복잡한 기호 대신 메서드를 사용하면 코드가 더 읽기 쉬워집니다.

`.isin([List])`

여러 값 중 하나 (OR 대체)

`.between(a, b)`

범위 확인 (AND 대체)

boolean_indexing.py

```
1 df = sns.load_dataset('titanic')
2 # 1. 단일 조건 마스크 생성 (성인 여부)
3 # 비교 연산(>=) 결과는 True/False 시리즈입니다.
4 mask_adult = df['age'] >= 18
5 print("Mask(Top 3):", mask_adult.head(3).values)
6
7 # 2. 마스크를 이용한 필터링: df[mask]
8 df_adult = df[mask_adult]
9 print(f"Adult count: {df_adult.shape[0]}")
10
11 # 3. 결측치(NaN) 확인: isna()
12 mask_na = df['age'].isna()
13 print(f"Missing Ages: {mask_na.sum()}")
14
15 # 4. 결측치 제외 데이터 추출: notna()
16 # 나이 데이터가 있는 유효한 행만 선택합니다.
17 df_valid = df[df['age'].notna()]
18 display(df_valid.head(3))
19
```

OUTPUT / TERMINAL

→ python boolean_indexing.py

Mask(Top 3): [True True True]

Adult count: 601

Missing Ages: 177

🔍 Boolean Mask 원리:

조건이 참(True)인 행은 선택하고, 거짓(False)인 행은 버립니다. 결측치는 비교 연산 시 False로 처리되므로 주의가 필요합니다.

df_valid (Top 3) - 결측치 제외됨

	survived	sex	age	fare
0	0	'male'	22.0	7.250
1	1	'female'	38.0	71.283
2	1	'female'	26.0	7.925
... Filtered Result ...				

filtering_advanced.py

```
1 df = sns.load_dataset('titanic')
2 # 1. 논리 연산자: & (AND), | (OR), ~ (NOT)
3 # 주의: 각 조건은 반드시 괄호()로 감싸야 합니다!
4 mask_p1 = (df['pclass'] == 1)
5 mask_f = (df['sex'] == 'female')
6 target = df[mask_p1 & mask_f]
7
8 # 2. 유용한 메서드: .isin(), .between()
9 # .isin(['A', 'B']): A 또는 B인 값 필터링
10 mask_city = df['embark_town'].isin(['Cherbourg', 'Southampton'])
11 # .between(20, 39): 20 이상 39 이하 (양끝 포함)
12 mask_age = df['age'].between(20, 39)
13
14 # 3. 3개 이상 조건 조합하기
15 # 조건: 20~39세 여성이면서 1,2등석에 탑승한 승객
16 final_mask = mask_age & mask_f & (df['pclass'].isin([1, 2]))
17 result = df.loc[final_mask, ['sex', 'pclass', 'age']]
18 print(f"Filtered Count: {len(result)}")
19 display(result.head(3))
```

OUTPUT / TERMINAL

→ python filtering_advanced.py

Filtered Count: 71

⚠ 주의 (Warning)

Python의 `and`, `or` 키워드는 Boolean Indexing에서 사용할 수 없습니다.
반드시 비트 연산자 `&`, `|`, `~`를 사용하고, 조건을 괄호 `()`로 묶어주세요.

Result Head (Top 3):

	sex	pclass	age
1	'female'	1	38.0
3	'female'	1	35.0
9	'female'	2	14.0
... showing specific columns ...			

실습 2: 타이타닉 조건 필터링

1

2030 여성 승객 추출

20세~39세 사이의 **female** 승객만 필터링하고, **sex**, **age**, **pclass** 열만 확인하세요.



2

Top 5 부유한 시니어 남성

50세 이상 남성 중 요금(**fare**)이 가장 높은 5명을 뽑으세요. (내림차순 정렬)



3

복합 조건 & 결측 제외

승선도시가 **Cherbourg** 또는 **Southampton**이면서, 나이 정보가 있는 승객만 남기세요.



GOAL

나이, 성별, 도시 등 **복합 조건**으로 승객 데이터를 자유자재로 추출한다.

exercise_02.py

💡 Hint: `between(a,b)`, `isin([..])`, `notna()` | 다중조건은 괄호() 필수

```
1 import pandas as pd
2
3 # 1. 2030 여성: (조건1) & (조건2)
4 c1 = (df['sex'] == 'female') & df['age'].between(20, 39)
5 task1 = df.loc[c1, ['sex', 'age', 'pclass']]
6
7 # 2. 50세 이상 남성 Top 5
8 c2 = (df['sex'] == 'male') & (df['age'] >= 50)
9 # 내림차순 정렬 후 상위 5명
10 task2 = df[c2].sort_values('fare', ascending=False).head(5)
11
12 # 3. 도시 선택(isin) & 나이 있음(notna)
13 towns = ['Cherbourg', 'Southampton']
14 c3 = df['embark_town'].isin(towns) & df['age'].notna()
15 task3 = df[c3]
16 # 결과 확인
17 display(task1.head(), task2, task3.shape)
```



SECTION 03

데이터 가공과 컬럼 조작

분석의 깊이를 더하는 파생 변수 생성부터
데이터셋을 깔끔하게 정리하는 방법까지 다룹니다.



학습 목표

이번 섹션에서 다룰 핵심 내용입니다.

1

파생 컬럼 만들기

기존 데이터를 조합하거나 연산하여 새로운 통찰을 주는 파생 변수 (Feature)를 생성합니다.

2

컬럼 삭제와 정리 (drop)

drop 메서드를 사용하여 분석에 방해가 되거나 불필요한 데이터를 안전하게 제거합니다.

3

가독성 높이기 (rename)

rename을 활용해 컬럼 이름을 직관적이고 일관성 있게 변경하여 데이터의 이해도를 높입니다.

컬럼 추가: 파생 변수로 통찰 만들기

1

합산 (AGGREGATION)

sibsp

+

1

→

family_size

parch

2

비율 (RATIO)

fare

÷

family_size

→

fare_per_person

3

구간화 (BINNING)

age: 22



pd.cut

→

adult

age: 5

child

데이터의 가치를 높이는 3가지 패턴



가족 크기 (Family Size)

형제자매(`sibsp`)와 부모자녀(`parch`)에 본인을 더해, 혼자인지 가족과 함께인지 파악합니다.



1인당 요금 (Fare per Person)

단체 승객은 요금이 합산되어 기록될 수 있습니다. 총 요금을 가족 수로 나누어 실제 1인당 비용을 추산합니다.



연령대 구간화 (Age Group)

연속적인 나이 숫자를 '어린이', '청소년', '성인' 같은 의미 있는 범주로 묶어 생존율 패턴을 더 명확하게 만듭니다.

데이터 구조 정리: 삭제와 변경

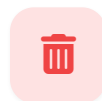


"원본은 지키고, 결과만 취한다"

판다스의 대부분 연산은 **비파괴적(Non-destructive)**입니다.

`inplace=True` 를 쓰지 않는 한,
항상 새로운 DataFrame을 반환합니다.

데이터 구조 정리 2단계



컬럼 삭제 (drop)

`axis=1`

분석에 불필요하거나 결측치가 너무 많은 열을 제거합니다.

A B C → A C



이름 변경 (rename)

`{old: new}`

직관적이지 않은 컬럼명을 이해하기 쉬운 이름으로 바꿉니다.

sex Map gender



안전한 코딩 습관

`inplace=True` 는 원본을 훼손하여 복구하기 어렵습니다.

대신 `df = df.drop(...)` 처럼 재할당하세요.

feature_engineering.py

```
1 import pandas as pd
2 # 1. 수치 연산: 가족 크기 (형제 + 부모 + 본인)
3 df['family_size'] = df['sibsp'] + df['parch'] + 1
4
5 # 2. 논리 연산: 14세 미만 어린이 여부 (True/False)
6 df['is_child'] = df['age'] < 14
7
8 # 3. 파생 변수: 1인당 요금 (전체 요금 / 가족 수)
9 df['fare_per_person'] = df['fare'] / df['family_size']
10
11 # 4. 구간화(Binning): 연령대 그룹 (pd.cut)
12 df['age_group'] = pd.cut( df['age'], bins=[0, 12, 18, 60, 100],
13                           labels=['child', 'teen', 'adult', 'senior'] )
14
15 # 결과 확인: 새로 만든 컬럼만 추출
16 new_cols = ['age', 'family_size', 'is_child', 'fare_per_person',
17             'age_group']
18
19 display(df[new_cols].head(3))
```

OUTPUT / TERMINAL

[→ python feature_engineering.py](#)

Success: Added 4 new columns to DataFrame

Preview (New Columns):

	age	family_size	is_child	fare_per_person	age_group
0	22.0	2	False	3.625	'adult'
1	38.0	2	False	35.641	'adult'
2	26.0	1	False	7.925	'adult'
... columns successfully created ...					

Insight

`pd.cut()`은 연속형 변수(나이)를 범주형(연령대)으로 변환할 때 매우 유용합니다.

`family_size`와 같은 파생 변수는 머신러닝 모델 성능을 높이는 핵심 요소입니다.

column_ops.py

```
1 import pandas as pd
2 # 1. 컬럼 삭제 (drop): 비파괴적 연산 (새 객체 반환)
3 # 'deck' 컬럼 제거. axis=1 또는 columns=[] 사용
4 df2 = df.drop(columns=['deck'])
5
6 # 2. 컬럼 이름 변경 (rename): {기존: 신규}
7 # 'sex' → 'gender', 'pclass' → 'class_num'
8 df3 = df.rename(columns={ 'sex': 'gender', 'pclass': 'class_num' })
9
10 # [미니 실습] 체이닝으로 한 번에 처리하기
11 # 1) 'alive' → 'survival_text' 변경
12 # 2) 'adult_male' 컬럼 삭제
13 df_final = df.rename(columns=
14     {'alive': 'survival_text'}).drop(columns=['adult_male'])
15
16 # 결과 확인 (컬럼 목록 비교)
17 print("Renamed Cols:", df_final.columns[-2:])
18
```

OUTPUT / TERMINAL

[→ python column_ops.py](#)

Step 1: Drop 'deck'

Original Columns: 15

Dropped Columns: 14 ('deck' removed)

Step 2: Rename 'sex' & 'pclass'

['survived', 'class_num', 'gender', 'age', ...]

PRACTICE RESULT

Chain Operation

Deleted:

adult_male

Renamed:

alive → survival_text

> print(df_final.columns[-2:])

Index(['embark_town', 'survival_text'], dtype='object')

SECTION 04

데이터 정렬

순서를 바꾸면 패턴이 보입니다.
원하는 기준으로 데이터를 나열하는 법을 배웁니다.



학습 목표

이번 섹션에서 다룰 핵심 내용입니다.

1

값 기준 정렬 (sort_values)

특정 컬럼의 값을 기준으로 오름차순 또는 내림차순으로 정렬하여 데이터의 분포와 순위를 파악합니다.

2

인덱스 기준 정렬 (sort_index)

데이터의 고유 식별자인 인덱스를 기준으로 정렬하여 데이터의 원래 순서나 시간 순서를 복원합니다.

3

다중 컬럼 및 결측치 제어

여러 컬럼을 동시에 정렬하고, 결측치(NaN)의 위치(맨 위/맨 아래)를 지정하는 고급 정렬 기법을 익힙니다.

정렬: 순서를 바꾸면 패턴이 보인다

Raw Data

2	Class B	25
0	Class A	40
1	Class A	30

▼ `sort_values(['Class', 'Age'])`

Sorted

0	Class A	40
1	Class A	30
2	Class B	25

Index 1차 정렬 (그룹) 2차 정렬 (값)

"기준이 여러 개일 땐 순서대로"

첫 번째 기준으로 줄을 세우고,
같은 값끼리는 두 번째 기준으로 다시 정렬합니다.

데이터 정렬의 3가지 핵심 원리



값 기준 정렬 (sort_values)

특정 컬럼의 데이터를 기준으로 오름차순(기본) 또는 내림차순 정렬합니다.

`ascending=False` (내림차순)



다중 조건 정렬 (Multi-column)

여러 컬럼을 리스트로 전달하여 복합적인 순서를 만듭니다.

예: "1등석 승객 중에서(1차), 나이가 많은 순서대로(2차)"



인덱스 복구 및 결측치 제어

`sort_index()`: 섞여버린 데이터의 순서를 행 번호 기준으로 복구

`na_position`: 결측치(NaN)를 맨 위('first')나 아래('last')로 배치

sorting.py

```
1 # 1. 단일 컬럼 정렬: 나이가 많은 순서대로 (내림차순)
2 df_age = df.sort_values('age', ascending=False)
3
4 # 2. 다중 컬럼 정렬: 등급(1->3)순, 같은 등급 내에선 나이(많->적)순
5 # pclass: 오름차순(True), age: 내림차순(False) df_multi =
6 df.sort_values(['pclass', 'age'], ascending=[True, False])
7
8 # 3. 결측치(NaN) 위치 제어: 나이 결측치를 맨 뒤로(last) 보냄
9 df_na = df.sort_values('age', na_position='last')
10
11 # 4. 인덱스 기준 정렬: 원래 순서 등으로 복구할 때 사용
12 df_idx = df.sort_index()
13
14 # 결과 확인 (다중 정렬 상위 5개)
15 print("--- Multi-Column Sorting Result (Top 3) ---")
16 display(df_multi[['pclass', 'sex', 'age', 'fare']].head(3))
17
18
```

OUTPUT / TERMINAL

→ python sorting.py

--- Multi-Column Sorting Result (Top 3) ---

Criteria: Pclass (Ascending) → Age (Descending)

	pclass	sex	age	fare
630	1	'male'	80.0	30.000
96	1	'male'	71.0	34.654
493	1	'male'	71.0	49.504
(Rows are sorted by Pclass=1 first, then Age desc)				

Note:

- `ascending=[True, False]` applies different sort directions to each column.
- Original Index (630, 96...) is preserved unless `ignore_index=True` is used.

종합 실습: 타이타닉 생존자 분석

MISSION GOAL

실제 분석 시나리오를 통해 **필터링**, **선택**, **정렬** 을 종합적으로 응용한다.

1

타겟 승객군 추출 (Filtering)

두 그룹을 각각 찾아 변수에 저장하세요:

- 1) 20~39세 여성 (df20f)
- 2) 50세 이상 남성 (df50m)



2

조건부 정렬 및 상위 추출 (Sort)

여성이면서 1, 2등석 승객 중(나이 결측 제외), 요금이 비싼 순서대로 상위 10명을 추출하세요.



3

생존률 패턴 분석 (Aggregation)

`sex` 와 `pclass` 별 생존률 평균을 구하고, 생존률이 **높은 순서대로** 정렬하여 패턴을 확인하세요.



final_mission.py

💡 **Tips:** 복잡한 조건은 ()로 감싸기 • `.isin()` 활용 • 정렬 후 `.head()` 사용

```
1 import seaborn as sns
2 df = sns.load_dataset('titanic')
3
4 # Mission 1. between & boolean indexing
5 mask_20f = (df['sex']=='female') & df['age'].between(20, 39)
6 df20f = df.loc[mask_20f]
7
8 # Mission 2. isin, notna, sort_values (Compressed)
9 mask = (df['sex']=='female') & df['pclass'].isin([1,2]) & df['age'].notna()
10 top10 = df[mask].sort_values('fare', ascending=False).head(10)
11
12 # Mission 3. groupby & sort
13 rate = df.groupby(['sex', 'pclass'])['survived'].mean()
14 result = rate.sort_values(ascending=False)
15 # Result Check
16 display(df20f.shape, top10, result.head())
```





Indexing & Slicing

loc (Label) vs iloc (Position)

```
df.loc['row_name', 'col_name'] // 끝 포함  
df.iloc[row_idx, col_idx] // 끝 제외
```

Avoid Chain Indexing

```
df[cond]['col'] = 1  
✓ df.loc[cond, 'col'] = 1
```



Column Operations

Create & Modify

```
df['new_col'] = df['a'] + df['b']  
df['category'] = pd.cut(df['age'], bins=3)
```

Drop & Rename

```
df = df.drop(columns=['unnecessary'])  
df = df.rename(columns={'old': 'new'})
```



Boolean Filtering

Logical Operators (Parentheses Required)

```
mask = (df['age'] > 20) & (df['sex'] == 'f')  
df[mask] or df.loc[mask, :]
```

Helper Methods

```
.isin([a, b]), .between(10, 20), .isna()
```



Sorting

Sort Values

```
df.sort_values('col1', ascending=False)  
df.sort_values(['c1', 'c2'], ascending=[True, False])
```

Sort Index

```
df.sort_index(axis=0) // Row sorting
```